

Title: MatchFlix

Contributors: Bhargav Sairam(Github:SairamBhargav), Karson Kelley(Github:KarsonKelley), Charlie Guo(Github:Walker-85002), Ryan Fayock(Github:RF12598), Aswaanth Karuppasamy(Github:AswaanthK), and Neville Jijina(Github:Neville-Jijina)

Project Description:

MatchFlix is an AI powered movie discovery platform designed to simplify the process of finding great films to watch. Our goal is to remove the stress and endless scrolling that often comes with choosing a movie. After creating a secure account, users can personalize their profile and begin exploring movie recommendations right away.

MatchFlix features an intuitive swipe based movie discovery page, where users can like or dislike personalized suggestions. By collecting a user's responses, our system continuously learns their tastes and tailors future recommendations to better match their preferences. Liked movies are stored in a dedicated Favorites page, giving users easy access to films they plan to watch later.

Our application integrates with The Movie Database (TMDB) API to provide real time movie data including posters, ratings, and descriptions. We also incorporate Gemini AI to enhance recommendations beyond traditional filtering, helping users discover movies they might not find on their own.

The platform is built using Node.js, Express, PostgreSQL, and Docker, and follows modern version control and testing practices through GitHub. Future enhancements include collaborative group features like shared swipe sessions and real time matching, expanding MatchFlix into a more social and interactive viewing experience.

Project Tracker - GitHub project board:

Video: https://youtu.be/ApH_eBKuGk8

VCS:

<https://github.com/CU-CSCI3308-Fall2025/group-project-SairamBhargav>

PROJECT BOARD:

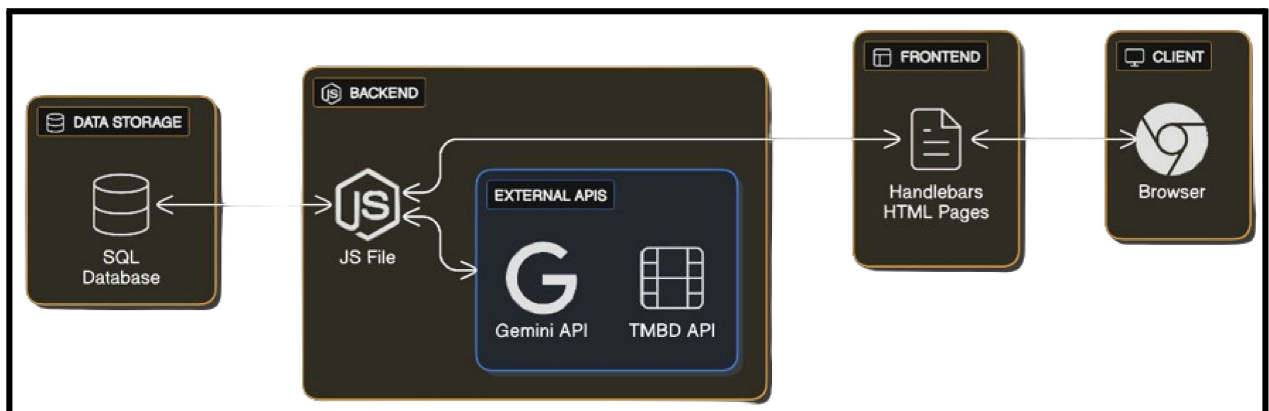
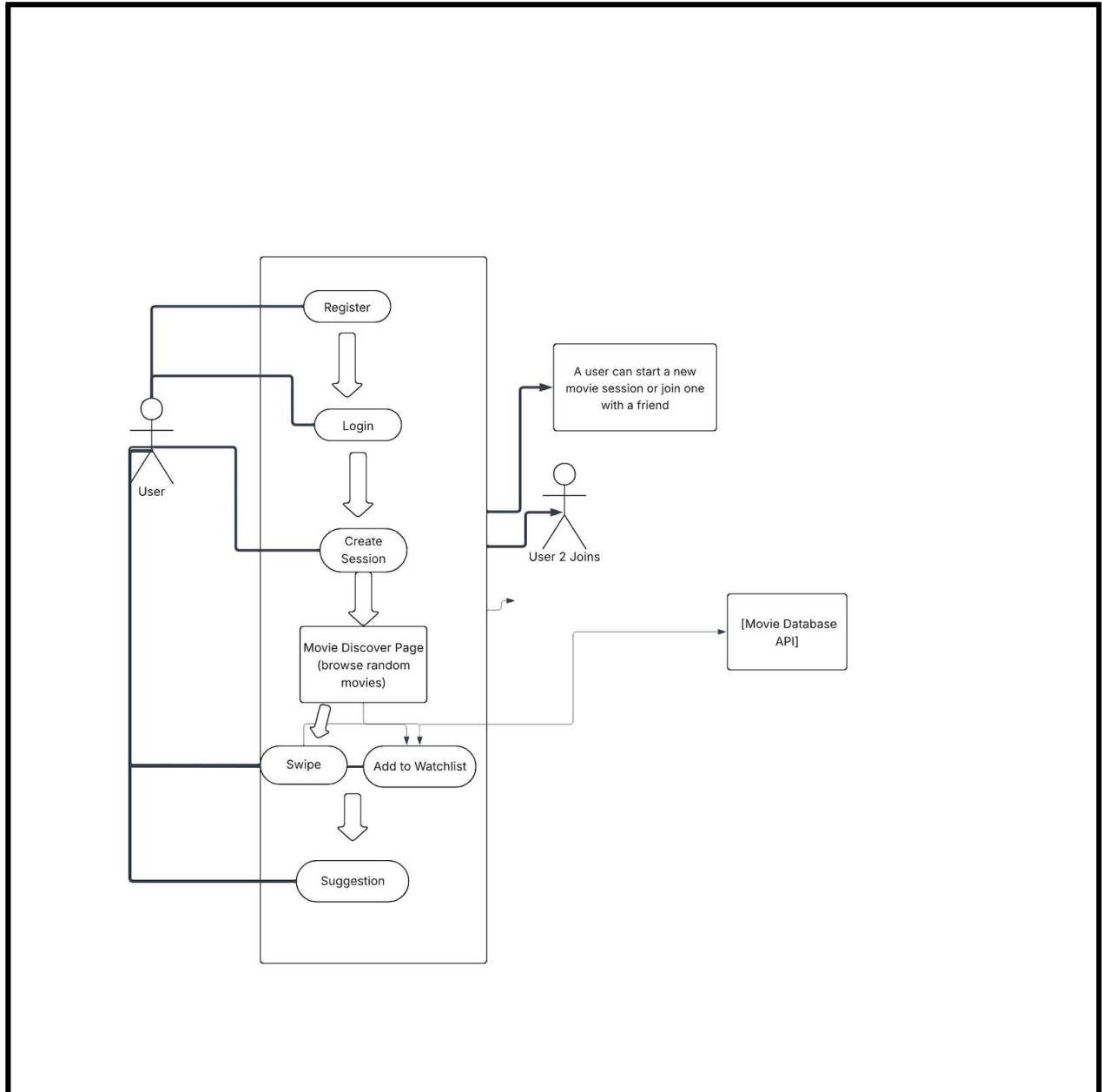
<https://github.com/orgs/CU-CSCI3308-Fall2025/projects/27>

Contributions:

- **Bhargav:**
 - I developed the user registration page including form validation and secure account creation. I contributed to backend routing and database integration to ensure smooth communication between the client and server. I also designed and implemented the AI powered movie recommendation query using Gemini and TMDB data to personalize results based on user preferences. Additionally, I supported testing and debugging to ensure reliable deployment on Render.
- **Karson:**
 - I designed the discover page, pulling from our TMDB API, I was able to display several new releases, their description, rating, and release date. On this discovery page there was also a feature to like and dislike movies. I also made the test cases to help debug and prevent the program from breaking in the future. And finally I setup render to deploy our full stack application on the web.
- **Charlie:**
 - I designed and implemented the liked page, which displays all the movies that a user has liked. This also involved helping with setting up the database in an efficient way, and a way that makes sense for our developments and future expansions.
- **Ryan:**
 - I developed the navbar to allow the user to change between the discover page, the swipe page, the liked page, the profile page, and the logout button. I also did the backend for the Swipe page, loading a new movie on each swipe, and properly pulling the API to display the movie
- **Aswaanth:**
 - I developed the user Profile page. This page tracks the total number of swipes, favorite actor, actress, and favorite genre. I did the front end html for this page as well as the backend routing and added a logout feature to it. I also helped with the html for the discovery.
- **Neville:**
 - I developed the user login page ensuring that the correct username was entered as well as the password. I did the front end and helped the routing for our swipe feature, to include a movie, description, rating, like button, dislike button, and a skip button with animations included. Also routed the likes and dislikes to our database

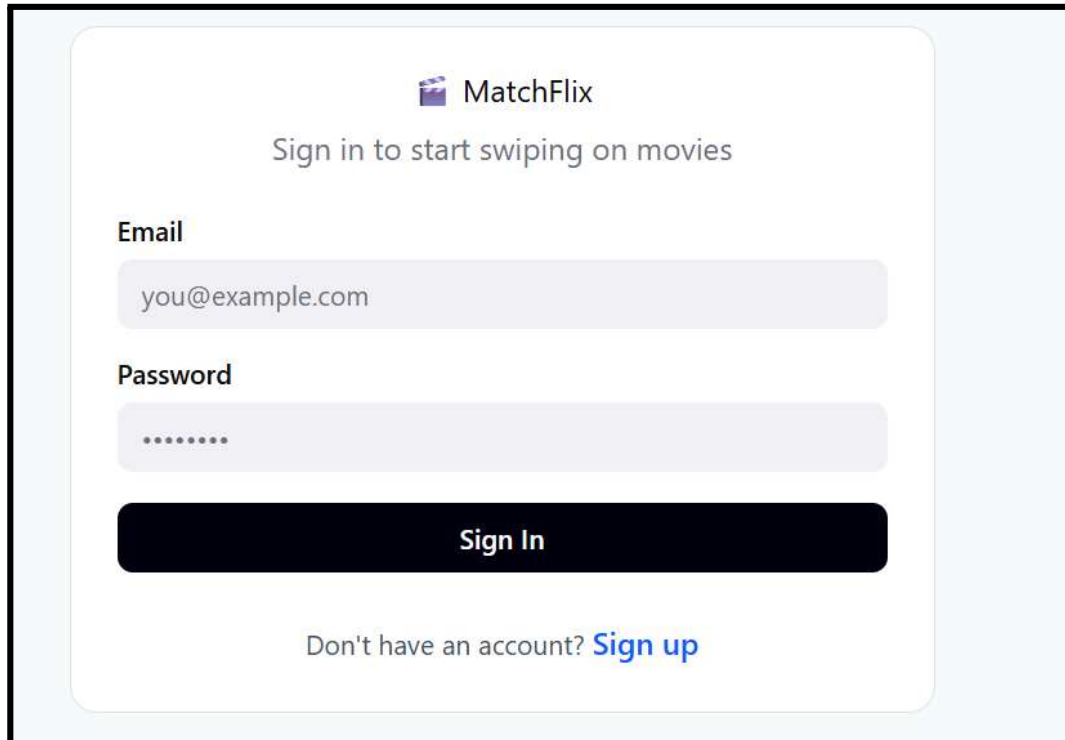


Use Case Diagram:



Wireframes:

Login page:



A wireframe of a login page for MatchFlix. The page is enclosed in a light blue border. At the top center is the MatchFlix logo, consisting of a purple and blue icon followed by the text "MatchFlix". Below the logo is the text "Sign in to start swiping on movies". There are two input fields: "Email" with the placeholder text "you@example.com" and "Password" with placeholder dots. Below these fields is a dark blue "Sign In" button. At the bottom, there is a link that says "Don't have an account? Sign up".

MatchFlix

Sign in to start swiping on movies

Email

you@example.com

Password

.....

Sign In

Don't have an account? [Sign up](#)

Profile Page:

← Back

 MatchFlix



Movie Lover
Movie Enthusiast

Total Swiped

 0

Liked

 0

Passed

 0

 Liked Movies (0)



No liked movies yet
Start swiping to build your collection!

Register Page:

 MovieMatch

Create an account to discover movies

Name

John Doe

Email

you@example.com

Password

.....

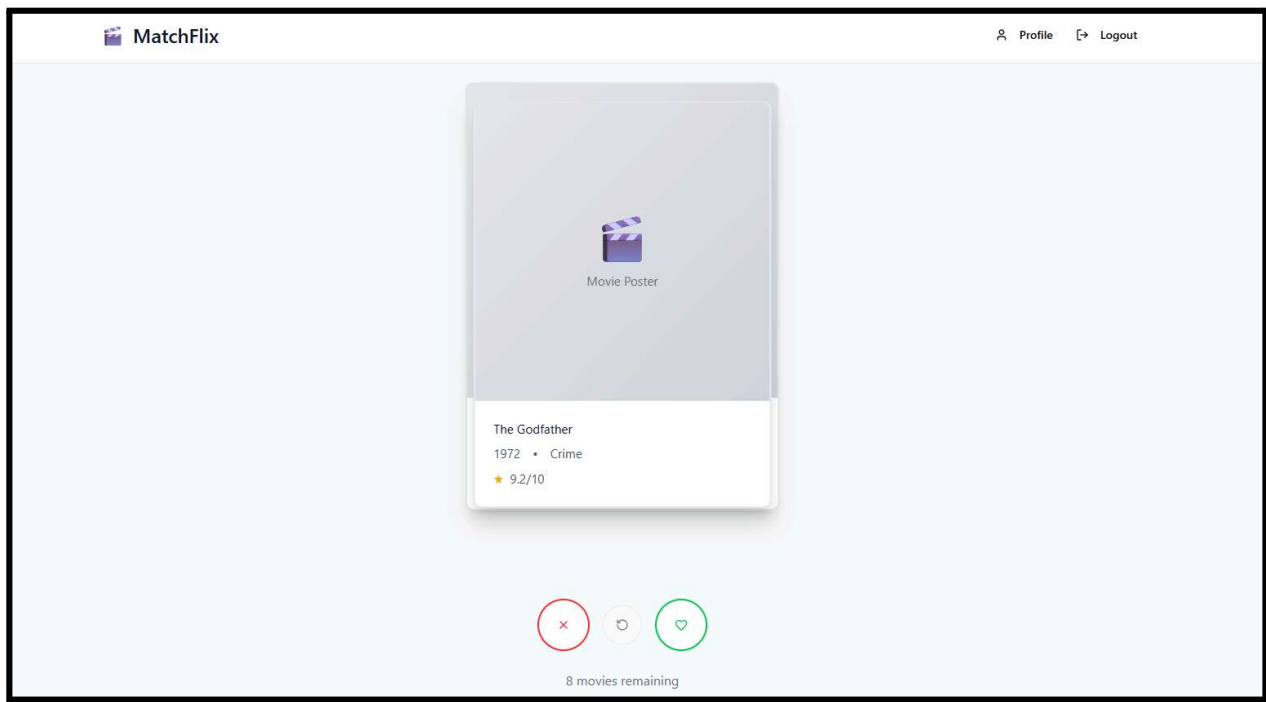
Confirm Password

.....

Create Account

Already have an account? [Sign in](#)

Swipe Page:



Test results:

Registration Page

Test Scenario

User is redirected to login page after registration

Test Cases

- Username: abc
- Password: 123
- First Name: Homer
- Last Name: Simpson
- Email: email@gmail.com
- Date of Birth: 01/01/2000

Test Environment

To test our registration page, navigate to <http://localhost:3000/register>

Testers

2 team members who are not the primary developers of the registration feature. Ideally, one technical user and one non-technical user to simulate real user behavior. Will check different variations of complete and incomplete fields.

Expected Test Results

When a user successfully registers, they will be directed to the login page. If they do not enter a valid username, password, first name, last name, email, and DOB, then they will be prompted with an error message.

Actual Test Results / Observations

All test cases passed successfully.

Tester 1 (Non-Technical User):

- Registered successfully with the test data on first try
- "The form was pretty straightforward. It told me what I missed when I forgot to fill in the email field."
- Tried submitting without entering the full date and got a clear error message
- "After registering, it just took me to login which made sense"

Tester 2 (Technical User):

- Tested edge cases like special characters in the password
- "I couldn't see my password in plain text anywhere which is good"
- Tried creating a duplicate account with username 'abc' and it properly rejected it
- "Validation works well, though it only checks when you hit submit"

Overall: Both testers successfully created accounts. The database stored credentials securely with hashed passwords. The redirect to login after registration worked smoothly.

Login Page

Test Scenario

When user enters a correct username and password, redirect them to the discover page

Test Case

- Username: abc
- Password: 123

Test Environment

To test our login page, navigate to <http://localhost:3000/login>

Testers

2 team members who did not make the login page can access the login page after registering to make sure they can enter a proper username and password and be redirected to the discover page.

Expected Test Results

User is able to login based on registered username and password and is redirected to the discover page. If info is wrong, they will receive a login error.

Actual Test Results / Observations

Login functionality worked as expected.

Tester 1:

- Logged in successfully with username 'abc' and password '123'
- "The redirect to discover page was instant, no lag"
- Tested wrong password and got "Incorrect username or password" error
- "I like that it doesn't tell you which one was wrong - seems more secure"

Tester 2:

- Tried logging in with an account that doesn't exist and got proper error
- "Session stayed active when I went to other pages, didn't have to login again"
- "Error messages are simple and clear"

Overall: Login works smoothly. Invalid credentials show appropriate errors without giving away too much info. Session management keeps users logged in as they navigate the site.

Movie Discover Page

Test Scenario

User can browse the discover page to see a list of several movies, their rating, and description

Test Case

- Logged-in user: Any registered user with valid session
- API: TMDB API (requires valid API key in .env file)
- Expected movie count: At least 10-20 popular movies

Test Environment

Local Host (<http://localhost:3000/discover>)

Testers

2 team members who didn't develop the discover page can properly browse it and explore the movie catalog.

Expected Test Results

The user logs into their account and is directed to the discover page immediately. The page shows movies with names, images of posters, ratings, and genres, and they are displayed in an orderly and well-designed manner.

Actual Test Results / Observations

The discover page performed flawlessly.

Tester 1:

- Counted 18 movies on the page, all with posters, titles, ratings, and descriptions
- "The layout looks clean and the movies are all recent popular ones"
- Checked on mobile view and "the grid adjusted to one column which looked good"
- "Load time was fast, no broken images"

Tester 2:

- Verified all movie data was complete - no missing info
- "The ratings matched what I saw on Google for a few movies I checked"
- "Cards are well-spaced, easy to read through quickly"

- Refreshed the page and stayed logged in, movies reloaded fine

API & Performance:

- TMDB API worked consistently with no failed requests
- Page loaded in under 2 seconds
- Responsive design worked on both desktop and mobile screen sizes

Overall: Both testers found the discover page to be a good landing spot after login. The movie information was complete and accurate. Layout was clean and professional-looking. One tester mentioned "it's like browsing Netflix."

Deployment:

MatchFlix is deployed using Render, which hosts both the web application and the PostgreSQL database. Deployment is fully containerized and configured to automatically rebuild and restart the application when changes are pushed to the GitHub repository.

How to Access the Application:

Live App URL: <https://matchflix-backend-9ojk.onrender.com/login>

Deployment Details:

1. Hosted on Render Web Services
2. Backend uses Node.js with Express
3. Database hosted on Render PostgreSQL
4. Environment variables securely stored in Render dashboard
5. Dependencies installed and app started automatically using the Render build process.

Running the App Locally (if needed)

1. Clone the GitHub repository
2. Create a .env file with credential placeholders
3. Run the following:
 - a. npm install
 - b. npm run testandrun
 - c. Docker-compose up

4. Access the app at <http://localhost:3000>